

DATA SCIENCE & AI MASTERS 2026

DAY 3 - PYTHON INTERVIEW PREPARATION

QUESTION BANK

Conditionals • Loops • Functions | 140 focused questions

Day 3 scope	Conditionals, loops, and functions in Python
Plan position	Phase 1 - Programming Foundations Study Day 3
Question bank	140 questions 45 marked ★ MUST ASK
Method	Answer aloud -> predict code -> write code -> explain why

Priority rule: ★ MUST ASK questions are the first revision set. Standard questions provide broader coverage. Every answer should be understandable without memorized wording.

★ MUST ASK - HIGHEST-PRIORITY QUESTIONS

These 45 questions are the first set to master before an interview. They emphasize the concepts, distinctions, code reasoning, and design decisions most likely to expose gaps in Python fundamentals.

★ MUST ASK #1. Explain if, elif, and else and when to use each.

Interview note: Know the control-flow model and why an if/elif/else chain selects at most one branch.

★ MUST ASK #2. Explain == vs = vs is.

Interview note: Assignment, equality, and identity are different concepts and are frequent interview traps.

★ MUST ASK #3. Explain truthy and falsy values in Python.

Interview note: Know common falsy values and how objects behave in Boolean contexts.

★ MUST ASK #4. Explain short-circuit evaluation with and/or.

Interview note: Be able to describe why Python may not evaluate the right-hand side.

★ MUST ASK #5. What is the difference between multiple if statements and if/elif/else?

Interview note: Independent if statements can execute multiple branches; an elif chain selects one matching branch.

★ MUST ASK #6. What is a for loop actually iterating over?

Interview note: An iterable; Python obtains elements from it rather than requiring numeric indexing.

★ MUST ASK #7. What is the difference between for and while?

Interview note: for is iterable-driven; while is condition-driven.

★ MUST ASK #8. Explain break, continue, and pass.

Interview note: break exits, continue skips one iteration, and pass does nothing.

★ MUST ASK #9. What is an infinite loop and how do you prevent one?

Interview note: Ensure the loop state can eventually make the termination condition false or provide a controlled break.

★ MUST ASK #10. What is range(start, stop, step) and why is stop excluded?

Interview note: Know the boundaries and how to count forwards/backwards without off-by-one errors.

★ MUST ASK #11. What is enumerate() and why is it preferable to manual counters?

Interview note: It provides index-value pairs directly and keeps loop intent clear.

★ MUST ASK #12. What does zip() do and what happens with unequal lengths?

Interview note: It pairs elements positionally and stops at the shortest input by default.

★ MUST ASK #13. What is loop-else and when does it execute?

Interview note: It runs after normal loop completion, but not after break.

★ MUST ASK #14. What is a function and why do we use functions?

Interview note: Reusability, organization, testing, readability, and maintainability.

★ MUST ASK #15. Parameter vs argument - explain with an example.

Interview note: A parameter is in the definition; an argument is supplied in the call.

★ MUST ASK #16. Explain return vs print.

Interview note: return provides a value to the caller; print only displays text/value.

★ MUST ASK #17. What does a function return if there is no return statement?

Interview note: None.

★ MUST ASK #18. Explain positional vs keyword arguments.

Interview note: Positional arguments match by order; keyword arguments match by parameter name.

★ MUST ASK #19. What are default arguments?

Interview note: Parameters that use a specified default when no argument is passed.

★ MUST ASK #20. What are *args and **kwargs?

Interview note: *args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dictionary.

★ MUST ASK #21. Explain argument unpacking with * and **.

Interview note: Use * to unpack an iterable into positional arguments and ** to unpack a mapping into keyword arguments.

★ MUST ASK #22. Explain local, global, and enclosing scope.

Interview note: Understand Python's name resolution and why local names normally do not modify global names.

★ MUST ASK #23. Explain the LEGB rule.

Interview note: Python searches Local, Enclosing, Global, then Built-in scopes.

★ **MUST ASK #24. What is a pure function and why is it useful?**

Interview note: Same inputs produce the same outputs without observable side effects; this improves testability and reasoning.

★ **MUST ASK #25. What are side effects in functions?**

Interview note: External observable changes such as modifying shared state or performing I/O.

★ **MUST ASK #26. Can functions be passed as arguments or returned?**

Interview note: Yes. Functions are first-class objects in Python.

★ **MUST ASK #27. What is recursion and what is a base case?**

Interview note: Recursion is self-calling; the base case terminates the recursion.

★ **MUST ASK #28. What happens when recursion has no valid base case?**

Interview note: Python eventually raises RecursionError after exceeding its recursion depth.

★ **MUST ASK #29. What is a closure?**

Interview note: A nested function that retains access to variables from its enclosing scope.

★ **MUST ASK #30. What is a lambda and when would you use one?**

Interview note: A short anonymous function for simple expressions, often as a callback or key function.

★ **MUST ASK #31. Why should functions return values instead of only printing them?**

Interview note: Returned values can be composed, tested, stored, and used by other code.

★ **MUST ASK #32. What is an early return and why does it improve code?**

Interview note: It exits as soon as a result is known, often reducing nesting and clarifying control flow.

★ **MUST ASK #33. How would you explain a nested loop's complexity?**

Interview note: If both loops scale with n , the work is often $O(n^2)$; explain based on how many times the inner work runs.

★ **MUST ASK #34. How do you avoid off-by-one errors in loops?**

Interview note: Be explicit about bounds, remember range stop is excluded, and test the smallest/largest cases.

★ **MUST ASK #35. What happens if you modify a list while iterating over it?**

Interview note: You can skip elements or produce unexpected behavior; prefer safer iteration strategies.

★ **MUST ASK #36. What is a conditional statement in Python?**

Interview note: A conditional lets a program choose which block to execute based on whether a condition evaluates to True or False.

★ **MUST ASK #37. What is the basic syntax of an if statement?**

Interview note: Use if followed by a Boolean expression and an indented block.

★ **MUST ASK #38. What is the purpose of an else block?**

Interview note: It runs when the preceding if/elif conditions are all False.

★ **MUST ASK #39. What is elif used for?**

Interview note: It lets you test additional conditions after an if without nesting another if.

★ **MUST ASK #40. Can a condition use logical operators?**

Interview note: Yes. and, or, and not are commonly used to combine conditions.

★ **MUST ASK #41. What happens if multiple elif conditions are True?**

Interview note: Only the first True branch is executed.

★ **MUST ASK #42. Can an if statement be nested inside another if statement?**

Interview note: Yes, but excessive nesting can reduce readability; combine conditions when appropriate.

★ **MUST ASK #43. What is a Boolean expression?**

Interview note: An expression that evaluates to True or False.

★ **MUST ASK #44. What are truthy and falsy values?**

Interview note: Python treats some objects as False in Boolean contexts, such as None, 0, False, and empty containers; many others are truthy.

★ **MUST ASK #45. What is the difference between == and = inside a condition?**

Interview note: == compares values; = assigns a value.

Conditionals - Core Concepts

What is a conditional statement in Python?

Interview note: A conditional lets a program choose which block to execute based on whether a condition evaluates to True or False.

What is the basic syntax of an if statement?

Interview note: Use if followed by a Boolean expression and an indented block.

What is the purpose of an else block?

Interview note: It runs when the preceding if/elif conditions are all False.

What is elif used for?

Interview note: It lets you test additional conditions after an if without nesting another if.

Can a Python if condition contain a comparison expression?

Interview note: Yes. Examples include `x > 10`, `age == 18`, and `score <= 50`.

Can a condition use logical operators?

Interview note: Yes. and, or, and not are commonly used to combine conditions.

What happens when an if condition is False and there is no else?

Interview note: The block is skipped and execution continues with the next statement.

Can you have multiple elif blocks?

Interview note: Yes. Python evaluates them from top to bottom and executes the first matching branch.

What happens if multiple elif conditions are True?

Interview note: Only the first True branch is executed.

Can an if statement be nested inside another if statement?

Interview note: Yes, but excessive nesting can reduce readability; combine conditions when appropriate.

What is a nested conditional?

Interview note: A conditional statement placed inside another conditional block.

What is a Boolean expression?

Interview note: An expression that evaluates to True or False.

What are truthy and falsy values?

Interview note: Python treats some objects as False in Boolean contexts, such as None, 0, False, and empty containers; many others are truthy.

Why is bool() useful when studying conditionals?

Interview note: It shows how Python evaluates an object in a Boolean context.

What is the difference between == and = inside a condition?

Interview note: == compares values; = assigns a value.

Why is using = instead of == in a condition a problem?

Interview note: = is assignment syntax, not the equality test used in a comparison.

What is the difference between is and == in a condition?

Interview note: == checks equality; is checks object identity. For ordinary value comparisons, use ==.

Loops - for and while

What is a loop?

Interview note: A control structure that repeats a block of code.

What are the two main loop types in Python?

Interview note: for and while.

What is a for loop used for?

Interview note: Iterating over items in an iterable such as a list, string, tuple, set, dictionary, or range object.

What is a while loop used for?

Interview note: Repeating code while a condition remains True.

What is the key difference between for and while?

Interview note: for iterates over an iterable; while repeats based on a condition.

What does iteration mean?

Interview note: Processing or visiting elements one at a time in a repeated sequence of steps.

What is an iterable?

Interview note: An object that can provide its elements one at a time for iteration.

Give common iterables used with for loops.

Interview note: Lists, tuples, strings, sets, dictionaries, ranges, and many other iterable objects.

What does range() commonly provide in a loop?

Interview note: A range object representing a sequence of integers, commonly used for counting iterations.

Why does range(5) produce five iterations?

Interview note: It represents 0 through 4 because the stop value is excluded.

What is the syntax of range(start, stop, step)?

Interview note: start is included when reached, stop is excluded, and step controls the increment or decrement.

How do you iterate backwards with range()?

Interview note: Use a negative step, for example range(5, 0, -1).

What happens if the step in range() is zero?

Interview note: Python raises ValueError because an iteration cannot progress with a zero step.

What is an infinite while loop?

Interview note: A while loop whose condition never becomes False.

How can you stop an intentionally infinite while loop?

Interview note: Use a break condition or otherwise change the state so the loop can terminate.

What is the most common cause of an accidental infinite while loop?

Interview note: Failing to update the state used by the loop condition.

What does break do?

Interview note: Immediately exits the nearest enclosing loop.

What does continue do?

Interview note: Skips the rest of the current iteration and moves to the next iteration.

What does pass do inside a loop?

Interview note: It performs no action and acts as a placeholder statement.

What is the difference between break and continue?

Interview note: break ends the loop; continue skips only the current iteration.

Can break be used in nested loops?

Interview note: Yes; it exits only the nearest enclosing loop.

Can continue be used in nested loops?

Interview note: Yes; it affects only the nearest enclosing loop's current iteration.

Functions - Foundations and Parameters

What is a function?

Interview note: A reusable block of code that performs a specific task and can accept inputs and produce an output.

Why are functions important in Python?

Interview note: They improve reuse, readability, testing, organization, and maintainability.

How do you define a function in Python?

Interview note: Use `def` followed by the function name, parameter list, colon, and indented body.

How do you call a function?

Interview note: Write the function name followed by parentheses, supplying arguments as needed.

What is a parameter?

Interview note: A variable listed in a function definition that receives an argument when the function is called.

What is an argument?

Interview note: The actual value supplied to a function call.

What is the difference between a parameter and an argument?

Interview note: A parameter is part of the function definition; an argument is the value passed at call time.

What does `return` do?

Interview note: It sends a value back to the caller and immediately exits the current function.

What happens if a function reaches the end without `return`?

Interview note: It returns `None` implicitly.

What is the difference between `print` and `return`?

Interview note: `print` displays a value; `return` sends a value back to the caller for further use.

Why is `return` generally more useful than `print` inside reusable functions?

Interview note: Returned values can be stored, tested, combined, and used by other code.

Can a function return multiple values?

Interview note: Yes. In practice Python packages multiple returned expressions into a tuple.

What is a default parameter value?

Interview note: A value used when the caller does not supply that argument.

How do you define a default parameter?

Interview note: Use `parameter=value` in the function definition.

What is a keyword argument?

Interview note: An argument passed by explicitly naming the parameter, such as `f(x=10)`.

What is a positional argument?

Interview note: An argument matched to a parameter based on its position in the call.

Positional argument vs keyword argument - which is clearer?

Interview note: Keyword arguments can improve clarity, especially when functions have several parameters.

Can keyword arguments follow positional arguments?

Interview note: Yes, provided the function call respects Python's argument-order rules.

Can positional arguments follow keyword arguments?

Interview note: No; positional arguments cannot come after keyword arguments in a call.

What are `*args`?

Interview note: A mechanism that collects additional positional arguments into a tuple.

What are `**kwargs`?

Interview note: A mechanism that collects additional keyword arguments into a dictionary.

Why would you use `*args`?

Interview note: When a function should accept a variable number of positional arguments.

Functions - Interview Traps and Practical Reasoning

What is recursion?

Interview note: A function calling itself directly or indirectly to solve a problem in smaller subproblems.

What is the base case in recursion?

Interview note: The condition that stops recursive calls.

What happens without a valid recursive base case?

Interview note: The recursion may continue until Python raises a RecursionError.

When should you prefer iteration over recursion in Python?

Interview note: Often when recursion would be deep or unnecessary, because Python does not optimize tail recursion and recursion depth is limited.

What is a closure?

Interview note: A nested function that retains access to variables from its enclosing scope after the outer function has finished.

Why might a closure be useful?

Interview note: It can preserve state without using global variables.

What is a lambda function?

Interview note: A small anonymous function created with the lambda expression.

When is lambda appropriate?

Interview note: For short, simple functions, often passed as an argument to another function.

Why should lambda not be used for complex logic?

Interview note: It reduces readability; a named def function is usually clearer.

What is the difference between defining and calling a function?

Interview note: Defining creates the reusable function object; calling executes it.

Practical Code-Reasoning Questions

What is the output of: if 0: print('A') else: print('B')?

Interview note: B, because 0 is falsy.

What is the output of: print(3 if 5 > 2 else 7)?

Interview note: 3.

What is the output of: for i in range(3): print(i)?

Interview note: 0, 1, 2 on separate lines.

What is the output of: for i in range(1, 5, 2): print(i)?

Interview note: 1, 3.

What is the output of: for x in 'AI': print(x)?

Interview note: A then I.

What is the output of: for i in range(5): if i == 2: continue; print(i)?

Interview note: 0, 1, 3, 4.

What is the output of: for i in range(5): if i == 2: break; print(i)?

Interview note: 0, 1.

What is the output of: for i in range(3): pass; print(i)?

Interview note: 0, 1, 2. pass does not stop the loop.

What is the output of: x=0; while x<3: print(x); x+=1?

Interview note: 0, 1, 2.

What happens if x is never incremented in that while loop?

Interview note: It becomes an infinite loop.

What is the output of: for i, v in enumerate(['a','b']): print(i,v)?

Interview note: 0 a and 1 b.

What is the output of: for a,b in zip([1,2],[3,4]): print(a+b)?

Interview note: 4 then 6.

What does this function return: def f(x): print(x)

Interview note: None, because there is no return statement.

What does this function return: def f(x): return x*2?

Interview note: Twice the input value.

What is the output of: def f(x=5): return x; print(f())?

Interview note: 5.

What is the output of: def f(a,b=2): return a*b; print(f(3))?

Interview note: 6.

What happens here: def f(a,b): return a+b; f(a=1,2)?

Interview note: SyntaxError: a positional argument cannot follow a keyword argument.

What is collected by *args?

Interview note: A tuple of extra positional arguments.

What is collected by **kwargs?

Interview note: A dictionary of extra keyword arguments.

What is the output of: def f(*args): return len(args); f(1,2,3)?

Interview note: 3.

What is the output of: def f(kwargs): return kwargs['x']; f(x=10)?**

Interview note: 10.

What is the output of: def outer(): x=10; def inner(): return x; return inner?

Interview note: Calling the returned inner function can access x because of a closure.

Why is print(f()) sometimes printing None even when f prints something?

Interview note: The function's print displays output, but if it has no return statement, the call expression evaluates to None.

Why can an early return simplify a function?

Interview note: It can eliminate nested else branches and make each condition's action explicit.

DAY 3 FINAL INTERVIEW CHECKLIST

- Explain if/elif/else and chained conditions clearly.
- Predict Boolean outcomes and short-circuit behavior.
- Choose for vs while for a given problem and explain why.
- Use range(), enumerate(), zip(), break, continue and loop-else correctly.
- Explain nested-loop cost and common off-by-one errors.
- Define functions correctly and distinguish parameters from arguments.
- Explain positional, keyword, default, *args and **kwargs arguments.
- Distinguish return from print and explain implicit None.
- Explain scope and LEGB; discuss side effects and pure functions.
- Explain first-class functions, recursion, closures and lambda at interview level.
- Solve short output-prediction problems without running the code first.

Completion standard: Day 3 is interview-ready when you can answer the ★ MUST ASK set aloud, reason through the code questions before execution, and write small control-flow/functions solutions from a blank editor.